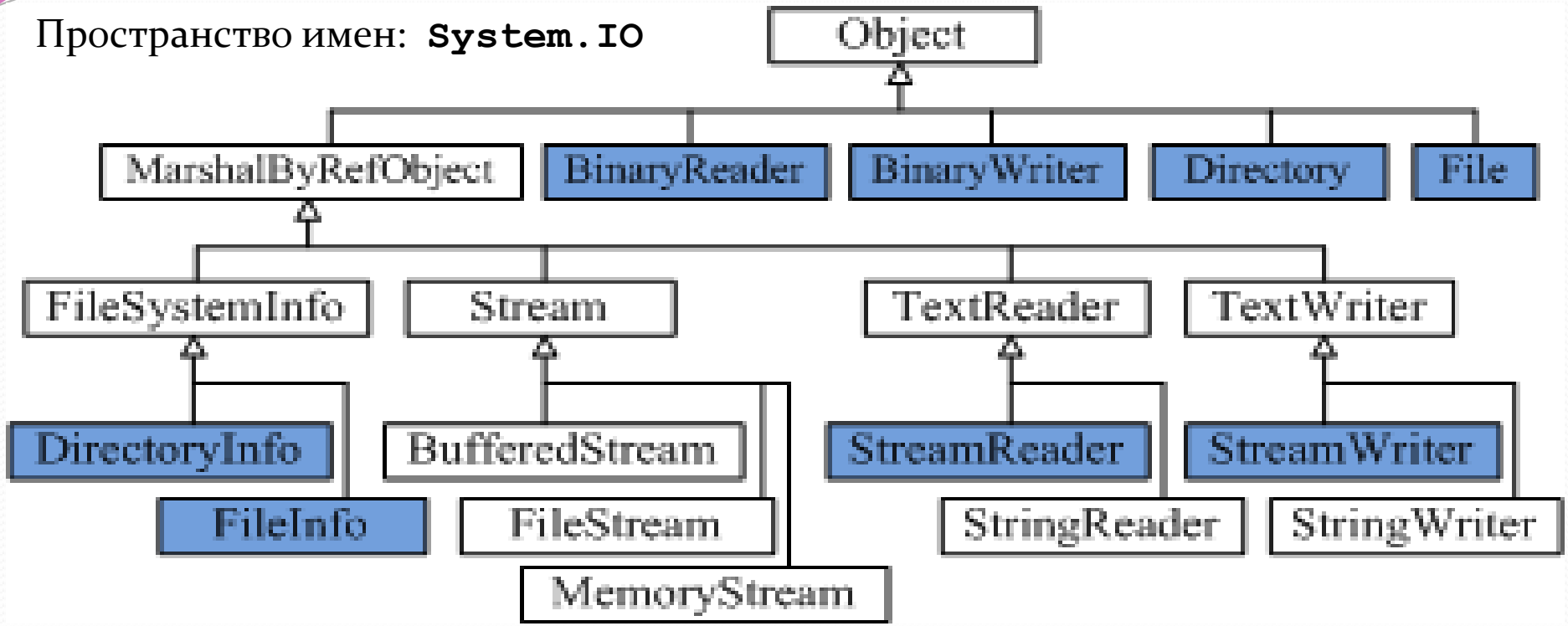
Глава 8.
Файлы в языке C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

8.1 Иерархия классов для работы с файлами

Пространство имен: **`System.IO`**



BinaryReader, BinaryWriter – работа на уровне двоичного представления данных – двоичные файлы;

FileStream – работа на уровне байтов;

StreamReader, StreamWriter – работа на уровне текста (строк) – текстовые файлы.

File – работа, и на уровне текста, и на уровне двоичного представления.

8.2 Каталоги и файлы

Статический класс Directory:

Дополнительно проверяет возможность выполнения операций.

Основные методы класса **Directory (вызываются с именем класса):**

- **CreateDirectory(path)**: создает каталог по указанному пути path
- **Delete(path)**: удаляет каталог по указанному пути path
- **Exists(path)**: определяет, существует ли каталог по указанному пути path. Если существует, возвращается true, если не существует, то false
- **GetCurrentDirectory()**: получает путь к текущей папке
- **GetDirectories(path)**: получает список подкаталогов в каталоге path
- **GetFiles(path)**: получает список файлов в каталоге path
- **GetFileSystemEntries(path)**: получает список подкаталогов и файлов в каталоге path
- **Move(sourceDirName, destDirName)**: перемещает каталог
- **GetParent(path)**: получение родительского каталога
- **GetLastWriteTime(path)**: возвращает время последнего изменения каталога
- **GetLastAccessTime(path)**: возвращает время последнего обращения к каталогу
- **GetCreationTime(path)**: возвращает время создания каталога

Класс DirectoryInfo

Основные методы класса **DirectoryInfo**:

- **Create()**: создает каталог
- **CreateSubdirectory(path)**: создает подкаталог по указанному пути path
- **Delete()**: удаляет каталог
- **GetDirectories()**: получает список подкаталогов папки в виде массива DirectoryInfo
- **GetFiles()**: получает список файлов в папке в виде массива FileInfo
- **MoveTo(destDirName)**: перемещает каталог

Основные свойства класса **DirectoryInfo**:

- **CreationTime**: представляет время создания каталога
- **LastAccessTime**: представляет время последнего доступа к каталогу
- **LastWriteTime**: представляет время последнего изменения каталога
- **Exists**: определяет, существует ли каталог
- **Parent**: получение родительского каталога
- **Root**: получение корневого каталога
- **Name**: имя каталога
- **FullName**: полный путь к каталогу

Пример Создание подкаталога

@ - обозначает дословный строковый литерал, т.е. позволяет не удваивать косые при записи пути

```
1. string path = @"C:\SomeDir";
   string subpath = @"program\avalon";
   DirectoryInfo dirInfo = new DirectoryInfo(path);
   if (!dirInfo.Exists)
   {
      dirInfo.Create();
   }
   dirInfo.CreateSubdirectory(subpath);
```

Создание объекта

```
2. string path = @"C:\SomeDir";
   string subpath = @"program\avalon";
   if (!Directory.Exists(path))
   {
      Directory.CreateDirectory(path);
   }
   Directory.CreateDirectory($"{path}/{subpath}");
```

Вызов метода с именем класса

Класс FileInfo

Некоторые полезные методы класса **FileInfo**:

- **CopyTo(path)**: копирует файл в новое место по указанному пути path
- **Create()**: создает файл
- **Delete()**: удаляет файл
- **MoveTo(destFileName)**: перемещает файл в новое место

Полезные свойства:

- Свойство **Directory**: получает родительский каталог в виде объекта DirectoryInfo
- Свойство **DirectoryName**: получает полный путь к родительскому каталогу
- Свойство **Exists**: указывает, существует ли файл
- Свойство **Length**: получает размер файла
- Свойство **Extension**: получает расширение файла
- Свойство **Name**: получает имя файла
- Свойство **FullName**: получает полное имя файла

Статический класс File

Класс File реализует похожую функциональность с помощью статических методов:

- **Copy()**: копирует файл в новое место
- **Create()**: создает файл
- **Delete()**: удаляет файл
- **Move**: перемещает файл в новое место
- **Exists(file)**: определяет, существует ли файл

8.3 Текстовые файлы
8.3.1 Класс File

- **`WriteAllText(string path, string? contents)`**: запись строки в текстовый файл (в ту же строку!). Если файл не существует, он создается, иначе - перезаписывается;
- **`void WriteAllLines(string path, string[] contents)`**: запись массива строк в текстовый файл. Если файл не существует, он создается, иначе – перезаписывается:
- **`AppendAllText(String path, String)`**: добавление в файл строки (к уже существующей строке без маркера-разделителя), если файл не существует, то он создается;
- **`AppendAllLines(string path, IEnumerable<String>)`**: добавление в файл набора строк. Если файл не существует, то он создается;
- **`string ReadAllText (string path)`** : чтение содержимого текстового файла в строку;
- **`string[] ReadAllLines(string path)`** : чтение содержимого текстового файла в массив строк;
- **`IEnumerable<string> ReadLines(string path)`** : чтение содержимого текстового файла в коллекцию строк.

Пример работы с текстовым файлом

Ex8_01

```
using System;
using System.IO;
namespace Ex8_01
{
  public class Program
  {
     static void Main(string[] args)
     {
        string path =
        @"/home/user/projects/Ex8_02/Ex8_02/content.txt";
         string originalText = "Hello Metanit.com";
         // запись строки
         File.WriteAllText(path, originalText);
         // дозапись в конец файла
         File.AppendAllText(path, "\nПривет, work");
         // чтение файла
         string fileText = File.ReadAllText(path);
         Console.WriteLine(fileText);
      }
   }
}
```

Вставка маркера Конец строки

Читается весь файл в одну строку!

Unicode. Кодировки UTF-8 и UTF-16

По умолчанию файл записывается в кодировке UTF-8 (~ANSI): Символы с кодами 0..127 кодируются одним байтом, остальные – 2..4 байтами:

content.txt — GHex

Файл Правка Вид Окна Справка

```
00000000 48 65 6C 6C 6F 20 4D 65 74 61 6E 69 74 2E 63 6F 6D 0A D0 Hello Metanit.com..
00000013 9F D1 80 D0 B8 D0 B2 D0 B5 D1 82 2C 20 77 6F 72 6B       ............, work
```

Символ "П"

Код UTF-8 $D09F_{16}$ => 110*10000 10*011111_2 => $100\ 0001\ 1111_2$

*красным отмечены служебные биты: 110 – код 16 бит, 10 – байт продолж.

Кодировка Unicode UNF-16 – каждому символу соответствует 16 или 32 бита.

Код Unicode: $1055_{10} = 41F_{16} = 100\ 0001\ 1111_2$

content.txt — GHex

Файл Правка Вид Окна Справка

```
00000000 FF FE 1F 04 40 04 38 04 32 04 35 04 42 04 20 00 4D 00 65 .....@.8.2.5.B. .M.e
00000013 00 74 00 61 00 6E 00 69 00 74 00 2E 00 63 00 6F 00 6D 00 .t.a.n.i.t...c.o.m.
00000026 0A 00 1F 04 40 04 38 04 32 04 35 04 42 04 2C 00 20 00 3C ....@.8.2.5.B.,. .<
00000039 04 38 04 40 04 21 00                                     .8.@.!.
```

Спецификатор UTF-16LE (начиная с младшего байта). Не обязателен.

Пример Работа с файлом в кодировке UTF-16 Unicode

Ex8_02

```
using System;
using System.IO;
using System.Text;
namespace Ex6_02
{
    public class Program
    {
        static void Main(string[] args)
        {
            string path =
                @"/home/user/projects/Ex8_02/Ex8_02/content.txt";
            string originalText = "Привет, Metanit.com";
            // запись строки
            File.WriteAllText(path, originalText,
                                                Encoding.Unicode);
            // дозапись в конец файла
            File.AppendAllText(path, "\nПривет, мир",
                                                Encoding.Unicode);
            // чтение файла
            string fileText = File.ReadAllText(path,
                                                Encoding.Unicode);
            Console.WriteLine(fileText);
        }
    }
}
```

8.3.2 Класс StreamWriter

Конструктор:

- `StreamWriter(string path[, bool append, [System.Text.Encoding encoding]]):` /* параметр append указывает на возможность добавления строк, параметр encoding указывает на кодировку, которая будет применяться при записи*/

Методы:

- `int Close();` //закрывает файл и освобождает все ресурсы
- `void Flush();` /*записывает в файл оставшиеся в буфере данные и очищает буфер */
- `void Write(string value);` /*записывает в файл данные простейших типов, как int, double, char, string и т.д. Перегружен для записи данных элементарных типов: Write(char value), Write(int value), Write(double value) и т.д.*/
- `void WriteLine(string value);` /*также записывает данные, только после записи добавляет в файл символ окончания строки*/

Пример

Ex8_03

```
string path =
  @"/home/user/projects/Ex6_03/Ex6_03/content.txt";
string text = "Hello, World!\nHello, Sun!";
// полная перезапись файла
StreamWriter writer = new StreamWriter(path,false);
writer.WriteLine(text);
writer.Close();
// добавление строк в файл
StreamWriter writer1 = new StreamWriter(path,true);
writer1.WriteLine("Addition");  // добавляет строку и "оА"
writer1.Write("4,5");           // добавляет строку
writer1.Flush();
```

Добавление запрещено, старый файл стирается.

content.txt — GHex

Файл Правка Вид Окна Справка

```
00000000 48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21 0A 48 65 6C 6C 6F Hello, World!.Hello
00000013 2C 20 53 75 6E 21 0A 41 64 64 69 74 69 6F 6E 0A 34 2C 35 , Sun!.Addition.4,5
```

8.3.3 Класс StreamReader

Конструкторы класса StreamReader:

- StreamReader(string path)
- **`StreamReader(string path, System.Text.Encoding encoding);`** //параметр encoding задает кодировку для чтения файла

Методы:

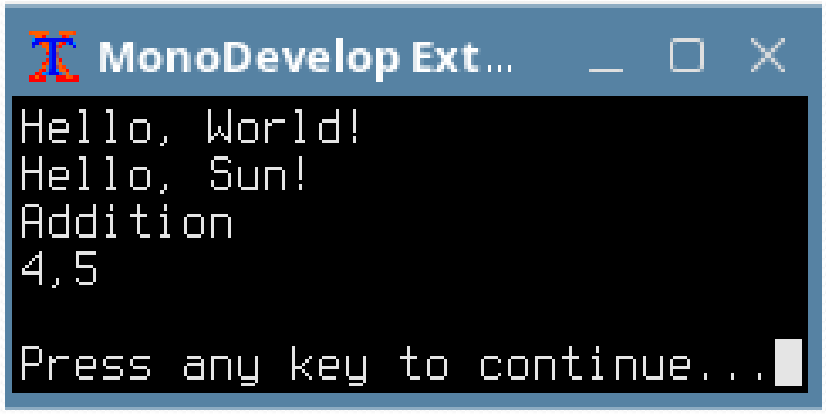
- **`void Close();`** /*закрывает считываемый файл и освобождает все ресурсы */
- **`int Peek();`** /*возвращает следующий доступный символ, если символов больше нет, то возвращает -1*/
- **`int Read();`** /*считывает и возвращает следующий символ в численном представлении. Имеет перегруженную версию: Read(char[] array, int index, int count), где array - массив, куда считываются символы, index - индекс в массиве array, начиная с которого записываются считываемые символы, и count - максимальное количество считываемых символов*/
- **`string ReadLine();`** //считывает одну строку в файле
- **`string ReadToEnd();`** //считывает весь текст из файла

Пример построчного чтения из файла

Ex8_04

```
string path =
  @"/home/user/projects/Ex8_04/Ex8_04/content.txt";

StreamReader reader = new StreamReader(path)
string line;
while ((line = reader.ReadLine()) != null)
{
    Console.WriteLine(line);
}
```



8.4 Двоичные файлы
8.4.1 Класс File

Чтение содержимое бинарного файла в массив байтов:

```
byte[] ReadAllBytes(string path)
```

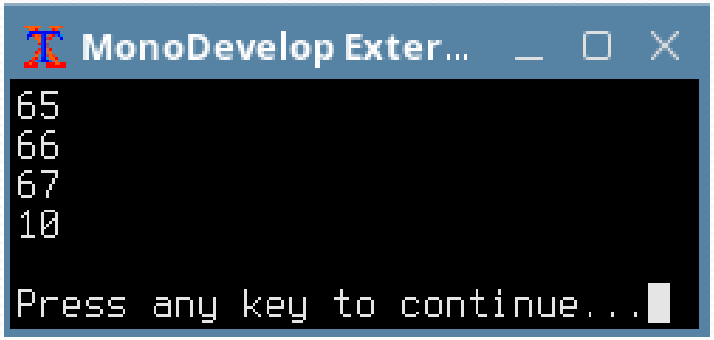
Запись массива байт в бинарный файл. Если файл не существует, он создается, иначе – перезаписывается:

```
void WriteAllBytes(string path, byte[] bytes)
```

Пример

Ex8_05

```
using System;
using System.IO;
namespace Ex8_05
{
class Program {
    public static void Main()
    {
        string path =
        @"/home/user/projects/Ex8_05/Ex8_05/content.txt";
        string[] text = {"ABC"};
        File.WriteAllLines(path, text);
        byte[] readText = File.ReadAllBytes(path);
        foreach(byte s in readText)
        {
            Console.WriteLine(s);
        }
    }
}
```



8.4.2 Класс BinaryWriter

Методы:

- **Close**(); закрывает поток и освобождает ресурсы
- **Flush**(): очищает буфер, дописывая из него оставшиеся данные в файл
- **Seek**(): устанавливает позицию в потоке
- **Write**(): записывает данные в поток. В качестве параметра этот метод может принимать значения примитивных данных: Write(bool), Write(byte), Write(String) и т.д.

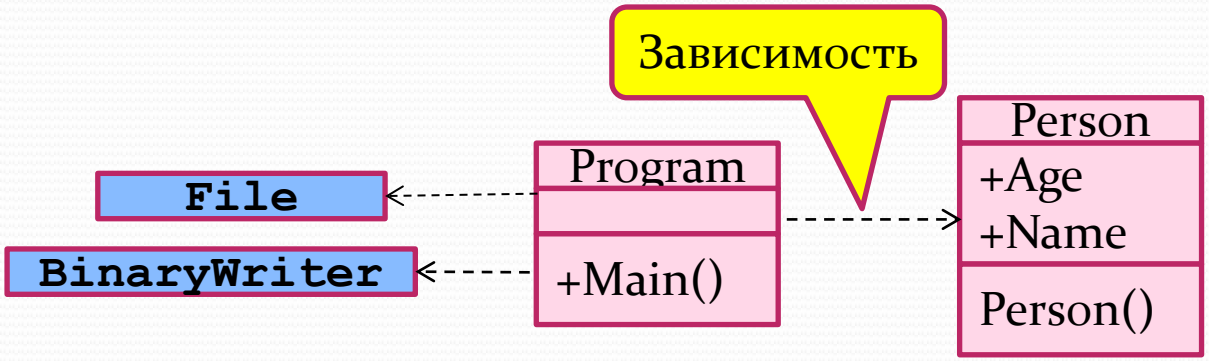
  Либо можно передать массивы типов byte и char

  - `Write(byte[])`
  - `Write(char[])`
  - `Write(ReadOnlySpan<byte>)`
  - `Write(ReadOnlySpan<char>)`

Пример записи двоичного файла

Ex8_06

```
using System;
using System.IO;
namespace Ex8_06
{
  class Person
  {
    public string Name { get; set; }
    public int Age { get; set; }
    public Person(string name, int age)
        {   name = name;  Age = age;  }
  }
```



Пример (окончание)

```
class Program {
   static void Main() {
      string path =
      @"/home/user/projects/Ex6_06/Ex6_06/content.txt";
      Person[] people =
          { new Person("Tom", 37),new Person("Bob", 41)};
      BinaryWriter writer = new
            BinaryWriter(File.Open(path,
                          FileMode.OpenOrCreate));
      foreach (Person person in people)
      {
          writer.Write(person.Name);
          writer.Write(person.Age);
      }
   }
 }
}
```

content.txt — GHex

Файл Правка Вид Окна Справка

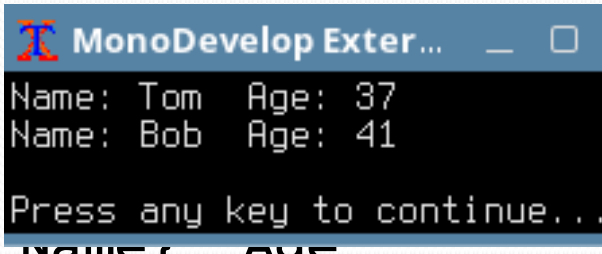
```
00000000 03 54 6F 6D 25 00 00 00 03 42 6F 62 29 00 00 00    .Tom%....Bob)...
```

8.4.3 Класс BinaryReader

Методы:

- **Close()**: закрывает поток и освобождает ресурсы
- **ReadBoolean()**: считывает значение bool и перемещает указатель на один байт
- **ReadByte()**: считывает один байт и перемещает указатель на один байт
- **ReadChar()**: считывает значение char, то есть один символ, и перемещает указатель на столько байтов, сколько занимает символ в текущей кодировке
- **ReadDecimal()**: считывает значение decimal и перемещает указатель на 16 байт
- **ReadDouble()**: считывает значение double и перемещает указатель на 8 байт
- **ReadInt16()**: считывает значение short и перемещает указатель на 2 байта
- **ReadInt32()**: считывает значение int и перемещает указатель на 4 байта
- **ReadInt64()**: считывает значение long и перемещает указатель на 8 байт
- **ReadSingle()**: считывает значение float и перемещает указатель на 4 байта
- **ReadString()**: считывает значение string. Каждая строка предваряется значением длины строки, которое представляет 7-битное целое число
- **PeekChar():** возвращает следующий символ

Пример

Ex8_07

```
List<Person> people = new List<Person>(); // список данных
string path =
     @"/home/user/projects/Ex6_07/Ex6_07/content.txt";
BinaryReader reader = new
                 BinaryReader(File.Open("people.dat",
                                        FileMode.Open));
while (reader.PeekChar() > -1)// пока не достигнут конец файла
 {
    string name = reader.ReadString();
    int age = reader.ReadInt32();
    people.Add(new Person(name, age));
 }
foreach(Person person in people)
 {
     Console.WriteLine($"Name: {person.Name}  Age:
                                        {person.Age}");
 }
```